

Лекція 4. GitLab. Огляд інструменту

GitLab – це сучасна платформа для управління життєвим циклом розробки програмного забезпечення, створена з урахуванням потреб DevOps-підходу. Вона забезпечує централізоване середовище, де всі учасники процесу розробки – від розробників і тестувальників до DevOps-інженерів і менеджерів – можуть ефективно співпрацювати. Платформа надає повний набір інструментів, що дозволяють автоматизувати, контролювати та вдосконалювати всі етапи розробки: від створення коду до його тестування, релізу та підтримки в продакшені.

Що таке GitLab?

GitLab - це веб-платформа для управління життєвим циклом програмного забезпечення. Вона містить у собі такі функції, як контроль версій, управління задачами, CI/CD, моніторинг та безпеку.



GitLab заснували Дмитро Запорожець та Сід Сібранджі в 2011 році. Ідея виникла через потребу в інструменті, який дозволяв би організовувати ефективну командну роботу над кодом із застосуванням системи контролю версій Git.

У 2011 році Дмитро Запорожець, український розробник, створив першу версію GitLab як проєкт з відкритим кодом для особистих потреб. Метою було спростити управління репозиторіями коду та взаємодію між членами команди.

У 2013 році до проєкту приєднався Сід Сібранджі, який допоміг комерціалізувати GitLab. Вони заснували компанію, щоб розвивати інструмент

як рішення для командної роботи. У 2015 році GitLab став платформою з інтегрованими функціями CI/CD.

У 2018 році компанія додала можливості DevSecOps, зосередившись на забезпеченні безпеки на всіх етапах розробки. З 2021 року GitLab став публічною компанією, вийшовши на біржу NASDAQ під тикером GTLB.

GitLab виділяється своєю філософією «все в одному»: замість використання багатьох різних інструментів для кожного етапу розробки, компанії отримують єдину інтегровану платформу, яка:

- спрощує управління вихідним кодом за допомогою вбудованої підтримки Git;
- забезпечує автоматизацію процесів розробки та розгортання через вбудовані пайплайни CI/CD;
- пропонує засоби для планування завдань, відстеження прогресу, управління пріоритетами та організації командної роботи;
- інтегрує інструменти безпеки для виявлення вразливостей ще до розгортання продукту;
- підтримує функції моніторингу, які дозволяють аналізувати продуктивність додатків у реальному часі.

GitLab також забезпечує масштабованість і гнучкість, що робить його придатним для використання як невеликими командами стартапів, так і великими корпораціями. Крім того, платформа може бути розгорнута як у вигляді хмарного сервісу (GitLab SaaS), так і в локальній інфраструктурі організацій (GitLab Self-Managed), що забезпечує високий рівень адаптації до різних потреб бізнесу.

Оскільки GitLab охоплює всі аспекти життєвого циклу розробки, він не лише спрощує робочі процеси, але й сприяє ефективнішій командній роботі, зменшує затрати часу на інтеграцію інструментів і допомагає досягати кращих результатів у коротші терміни.

Основні функції GitLab

- Контроль версій
- Управління задачами
- CI/CD
- Моніторинг
- Безпека



Основні функції GitLab:

- *управління вихідним кодом*: GitLab пропонує повноцінну підтримку системи контролю версій Git, що дозволяє розробникам:
 - створювати, зберігати та керувати репозиторіями коду;
 - використовувати розгалуження (branches) для ізоляції змін перед інтеграцією в основний код;
 - об'єднувати зміни за допомогою Merge Requests (MR), що дозволяє рецензувати код і коментувати його до внесення;
 - використовувати вбудовані інструменти для аналізу коду та перевірки якості перед інтеграцією;
 - надавати доступ до репозиторіїв на основі ролей (наприклад, розробник, менеджер, гість) для контролю рівня доступу;
- *CI/CD*: GitLab має вбудовані можливості для безперервної інтеграції та доставки (CI/CD), які дозволяють:
 - CI (Continuous Integration): Автоматично запускати тестування коду після кожного комміту, що допомагає знайти та виправити помилки на ранніх етапах;
 - CD (Continuous Delivery/Deployment): Автоматизувати процеси збірки, тестування та розгортання додатків;

- писати та налаштовувати GitLab CI/CD pipelines за допомогою YAML-файлів, що дозволяє гнучко контролювати процеси;
- підтримувати інтеграцію з контейнеризацією, використовуючи Docker та Kubernetes, для спрощення управління середовищами розгортання;
- отримувати сповіщення про статус виконання CI/CD-пайплайнів у реальному часі;
- *DevSecOps*: GitLab підтримує DevSecOps-підхід, що включає в себе інструменти безпеки на всіх етапах DevOps-процесу:
 - ***Static Application Security Testing (SAST)***: аналізує вихідний код для виявлення вразливостей;
 - ***Dynamic Application Security Testing (DAST)***: перевіряє програму під час її виконання для виявлення потенційних ризиків;
 - ***Dependency Scanning***: аналізує залежності (бібліотеки та пакети) для пошуку вразливих версій;
 - ***Container Scanning***: перевіряє контейнерні образи на вразливість;
 - ***Security Dashboard***: надає зведену інформацію про всі знайдені проблеми безпеки, допомагаючи їх відслідковувати та виправляти;
- *інструменти планування*: GitLab надає можливості для управління проектами:
 - ***Issues***: завдання та проблеми, які можна відстежувати, групувати за тегами та організовувати;
 - ***Epics***: інструмент для управління великими задачами, які охоплюють кілька Issues;
 - ***Roadmaps***: візуалізація дорожньої карти проекту, що допомагає визначати пріоритети та терміни виконання;
 - ***Milestones***: використовуються для групування задач за ключовими етапами розробки;
 - ***Boards***: Канбан-дошки для організації тасків, що дозволяє командам легко управляти прогресом виконання;

- *моніторинг та аналітика*: GitLab допомагає командам контролювати стан розробки та розгорнутих програм:
 - моніторинг продуктивності: інтеграція з інструментами типу Prometheus для збору метрик;
 - аналіз логів: допомагає виявляти проблеми в роботі додатку;
 - CI/CD-аналітика: звіти про швидкість виконання пайплайнів, частоту успішних/невдалих збірок;
 - DevOps Dashboard: центральний інструмент для моніторингу всіх DevOps-процесів команди.
- *інтеграції*: GitLab легко інтегрується з широким спектром зовнішніх інструментів:
 - керування проектами: Jira, Trello, Asana;
 - контейнеризація: Docker, Kubernetes для розгортання контейнерних додатків;
 - моніторинг: Prometheus, Grafana;
 - хмарні сервіси: AWS, Google Cloud, Microsoft Azure;
 - системи управління кодом: GitHub, Bitbucket (для міграції або інтеграції репозиторіїв).

Цей набір функцій робить GitLab універсальною платформою для командної розробки програмного забезпечення, що відповідає сучасним вимогам DevOps та DevSecOps.

Як GitLab відрізняється від Github?



GitLab та Github є подібними платформами для управління життєвим циклом програмного забезпечення. Однак GitLab дозволяє створювати приватні репозиторії, що робить його популярним серед комерційних проектів. Крім того, GitLab надає можливість розгортання на власному сервері та має відкритий вихідний код, що дозволяє розробникам вносити свої власні зміни та розширення до програмного забезпечення.

GitLab – це багатофункціональна платформа, створена для оптимізації всіх аспектів розробки програмного забезпечення. Її головне призначення полягає у забезпеченні ефективної співпраці, автоматизації процесів та централізації управління проєктами.

Налаштування системи GitLab

GitLab може бути встановлений на локальних серверах або у хмарі. Основні способи встановлення:

- *Omnibus-накет*: універсальне рішення, що включає все необхідне;
- *Docker*: для контейнеризації;
- *Kubernetes*: для оркестрації контейнерів;
- *Manual Installation*: встановлення компонентів вручну.

Процес встановлення (Omnibus) може містити наступні етапи:

1. додати сховище GitLab:

```
curl https://packages.gitlab.com/install/repositories/gitlab/gitlab-ee/script.deb.sh  
| sudo bash
```

2. встановити GitLab:

```
sudo EXTERNAL_URL="http://gitlab.example.com" apt-get install gitlab-ee
```

3. налаштувати конфігурацію згідно потреб: потрібно відредагувати файл `/etc/gitlab/gitlab.rb`. та налаштувати URL, порти, SSL.

4. запустити інструмент GitLab:

```
sudo gitlab-ctl reconfigure
```

Після встановлення потрібно перейти за URL (наприклад, <http://gitlab.example.com>) та увійти в систему під обліковим записом адміністратора (створений автоматично). Наступним кроком є налаштування глобальних параметрів (наприклад, SMTP для відправки листів, інтеграції тощо).

Для керування користувачами потрібно створити їх через веб-інтерфейс або API та обрати ролі користувачів з наступних категорій Owner, Maintainer, Developer, Reporter, Guest. Для зручності керування проекти та користувачі об'єднуються в групи.

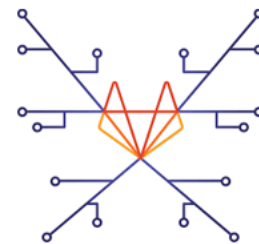
Основні інструменти GitLab

1. Репозиторії Git

GitLab використовує Git як основну систему контролю версій, забезпечуючи розподілену роботу з кодом. Це дозволяє кожному розробнику мати локальну копію всієї історії проекту. Користувачі можуть переглядати, редагувати та аналізувати код безпосередньо через веб-інтерфейс GitLab.

Контроль версій

GitLab дозволяє контролювати версії коду та зберігати його в репозиторії. Крім того, GitLab дозволяє створювати гілки для розробки функціональності та злити їх з головним кодом (merge).



2. Code Review

Code Review (перевірка коду) є ключовим етапом у розробці програмного забезпечення, метою якого є підвищення якості коду, забезпечення відповідності стандартам та виявлення помилок на ранніх етапах. У GitLab цей процес інтегрований у робочий цикл через **Merge Requests (MR)** та супутні інструменти.

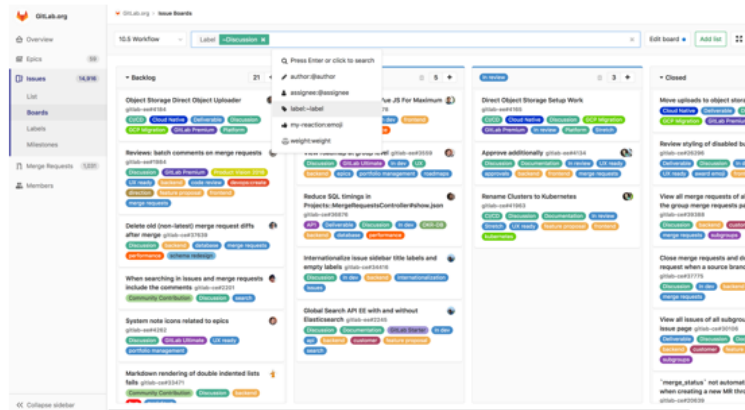
3. Issue Tracking

Issue Tracking – це система управління завданнями та відстеження проблем, інтегрована в GitLab. Вона дозволяє створювати, організовувати та

контролювати роботу над завданнями всередині проекту. Цей інструмент є важливим компонентом Agile-розробки, забезпечуючи прозорість і контроль на кожному етапі розробки.

Управління задачами

GitLab дозволяє створювати задачі та прив'язувати до них код. Він також надає можливість призначати відповідальних за рішення задачі та встановлювати терміни виконання.



4. Security & Compliance

Security & Compliance – це набір інструментів та функцій GitLab, спрямованих на забезпечення безпеки коду, управління ризиками та відповідності нормативним стандартам. Вони інтегровані в DevSecOps-підхід, що дозволяє командам забезпечувати безпеку на кожному етапі життєвого циклу розробки (SDLC).

5. Wiki

Wiki в GitLab – це інтегрований інструмент для створення та зберігання документації, пов'язаної з проектом. Він дозволяє командам організовувати знання, ділитися ними та підтримувати актуальність інформації в одному централізованому місці. GitLab Wiki базується на системі контролю версій Git, що забезпечує можливість відстеження змін і повернення до попередніх версій.

6. Моніторинг та Аналітика

Моніторинг та Аналітика у GitLab – це набір інструментів, які забезпечують аналіз продуктивності, виявлення проблем, а також збір метрик у реальному часі

для оцінки стану застосунків, інфраструктури й процесів розробки. Цей функціонал допомагає командам розробників і DevOps-інженерам приймати обґрунтовані рішення на основі даних.

Порівняння інструментів для роботи з системами контролю версій

GitLab, GitHub та Bitbucket, використовують Git як основу для управління вихідним кодом. Це забезпечує ключові можливості, які роблять роботу з кодом прозорою, надійною та організованою.

Зокрема, всі зазначені інструменти забезпечують:

- *відстеження змін*: розробники можуть легко переглядати історію змін, повертатися до попередніх версій або аналізувати коміти для розуміння еволюції проєкту;
- *робота з гілками (branches)*: кожна система дозволяє створювати окремі гілки для нових функцій, виправлення помилок чи експериментів. Це запобігає змішуванню незавершених змін із основною гілкою;
- *злиття змін (merge)*: усі ці платформи підтримують зручні інструменти для об'єднання змін, дозволяючи проводити рев'ю коду перед тим, як включити новий функціонал у основну версію програми.

Підтримка CI/CD

Усі згадані системи інтегрують або надають інструменти для безперервної інтеграції та доставки, що є основою сучасних DevOps-процесів.

GitHub: Використовує **GitHub Actions** – потужний інструмент для автоматизації робочих процесів. Це включає налаштування тестування, збірки та розгортання програм із широкою підтримкою сторонніх інтеграцій.

Bitbucket: Пропонує **Bitbucket Pipelines**, які забезпечують CI/CD із простим налаштуванням для хмарних репозиторіїв, але з деякими обмеженнями для локальних серверів.

GitLab: Має вбудовану підтримку CI/CD, яка дозволяє автоматизувати тестування, збірку та розгортання без необхідності сторонніх інструментів.

GitLab пропонує гнучкі налаштування пайплайнів через YAML-файли, а також інтеграцію з Docker і Kubernetes для роботи з контейнеризованими додатками.

CI/CD

Функція CI/CD в GitLab дозволяє автоматизувати процеси збирання, тестування та розгортання програмного забезпечення. GitLab CI/CD може запускати автоматичні тестові сценарії, компілювати програмний код, створювати збірки та забезпечувати їх розгортання в середовищі продакшену. Для цього використовується файл конфігурації '`gitlab-ci.yml`', в якому вказуються кроки для кожної стадії (будь-то збирання, тестування чи розгортання) та умови запуску цих стадій (наприклад, тільки після пушу в певну гілку). GitLab також підтримує інтеграцію з різноманітними інструментами для CI/CD, такими як Jenkins, Travis CI та інші.



Спільна робота над кодом

Платформи, пропонують широкий набір інструментів для командної роботи, що полегшує процес колаборації розробників, тестувальників і DevOps-фахівців:

- *Merge Requests* (аналог *Pull Requests*): інструмент для перегляду і об'єднання змін у коді. У GitLab Merge Requests містять усі необхідні компоненти, включаючи рев'ю, історію обговорення та статус тестування CI/CD. У GitHub: Pull Requests підтримують коментарі, рев'ю, інтеграцію з GitHub Actions для запуску тестів і перевірок. Вони включають шаблони для стандартизації запитів і інтеграцію з GitHub Discussions для тривалих обговорень. У Bitbucket: Pull Requests дозволяють створювати обговорення навколо змін у коді, з підтримкою коментарів, зазначення рев'юерів та автоматичних перевірок через Pipelines.
- *рев'ю коду* (*Code Review*): розробники можуть переглядати код своїх колег, залишати зауваження, пропонувати зміни та затверджувати готові коміти.

Цей процес допомагає покращити якість коду, зменшити кількість помилок і підвищити продуктивність команди;

- *обговорення та коментарі*: є змога коментувати конкретні рядки коду, обговорювати правки безпосередньо в Merge Requests, створювати тредовані дискусії та додавати роздільні коментарі для кращої організації роботи;
- гнучкі правила рев'ю: можна налаштувати обов'язковий список рев'юерів, що гарантує проходження змін кількома членами команди перед їхнім впровадженням.

Ці можливості роблять спільну роботу над кодом ефективною, особливо для розподілених команд.

Інтеграція з іншими інструментами

Всі наведені інструменти в певній мірі підтримують інтеграції з великою кількістю популярних сервісів і платформ, що дозволяє організаціям налаштовувати свої робочі процеси під конкретні потреби:

- *Jira*: GitLab має пряму інтеграцію, що дозволяє автоматично створювати посилання на Jira-завдання з комітів, Merge Requests та Issues. Інтеграція через GitHub Marketplace (наприклад, плагіни), дозволяє автоматично GitHub додавати посилання на Jira-завдання в коміти та Pull Requests. Bitbucket має найкращу інтеграцію серед інших платформ, оскільки Bitbucket є частиною Atlassian Suite. Прямий зв'язок із Jira-завданнями дозволяє переглядати статуси завдань і робити автоматичні оновлення через коміти та Pull Requests;
- *Slack*: нативна інтеграція GitLab через Slack App дозволяє надсилати повідомлення про статус пайплайнів, Merge Requests, Issues та інші події у Slack-канали. GitHub використовуючи інтеграцію через GitHub Slack App може надсилати повідомлення про Pull Requests, Issues, CI/CD-статуси та коміти у Slack-канали. Інтеграція Bitbucket через Bitbucket Slack App для сповіщень про Pull Requests, Issues та CI/CD статуси у Slack-канали;

- *Microsoft Teams*: GitLab підтримується через вебхуки або нативні конектори, які надсилають оновлення про зміни в коді, пайплайни та рев'ю. Нативна інтеграція GitHub через GitHub App для Microsoft Teams дозволяє надсилати сповіщення та обговорювати зміни в коді. Інтеграція Bitbucket через вебхуки або сторонні конектори дає можливість отримувати повідомлення про активність у репозиторіях;
- *Docker*: GitLab має глибоку інтеграцію, включаючи автоматичну побудову Docker-образів у CI/CD та публікацію їх у GitLab Container Registry. GitHub має підтримку створення Docker-образів через GitHub Actions. Образи можна зберігати в GitHub Container Registry. Bitbucket підтримує створення Docker-образів через Bitbucket Pipelines. Можна зберігати образи в Docker Hub або інших реєстрах;
- *Kubernetes*: нативна інтеграція дає можливість розгортати застосунки, здійснювати моніторинг та управляти кластерами Kubernetes безпосередньо через GitLab. Інтеграція GitHub через GitHub Actions дає можливість автоматичного розгортання на Kubernetes-кластери. Інтеграція Bitbucket через Pipelines розгортає контейнери у кластери Kubernetes;
- *Prometheus i Grafana*: GitLab має вбудовані функції моніторингу через Prometheus. Дані з Prometheus можна інтегрувати з Grafana для візуалізації. GitLab дозволяє відстежувати метрики CI/CD і застосунків. GitHub підтримує через сторонні інтеграції або GitHub Actions збір метрик і моніторинг CI/CD. Bitbucket можна інтегрувати через сторонні інструменти або вебхуки, але нативної підтримки немає;
- *інші сервіси*: інтеграція GitLab з Terraform для інфраструктури як коду, підтримка AWS, Google Cloud і Azure для розгортання, а також Webhooks та REST API для кастомних інтеграцій. GitHub має багатий Marketplace, що включає інтеграції з AWS, Google Cloud, Azure DevOps, Jenkins, Terraform, і численні Webhooks для кастомізації. Bitbucket підтримує глибоку інтеграцію з іншими Atlassian-продуктами, такими як Confluence,

Trello, Bamboo. Підтримка AWS, Google Cloud, Terraform та інших сервісів через Pipelines.

Завдяки цим інтеграціям команди можуть об'єднувати розрізнені інструменти в єдину екосистему, що значно підвищує продуктивність.

Управління проєктами

Всі розглянуті платформи мають вбудовані функції для управління проєктами, що дозволяють командам не лише писати код, але й планувати та організовувати свою роботу:

- *трекінг завдань*: у GitLab та GitHub є інструменти для створення та керування завданнями, коментувати їх, призначати відповідальних і відстежувати прогрес;
- *епіки та дорожні карти (Roadmaps)*: Функціонал дозволяє групувати завдання в більші епіки, створювати загальну картину проєкту та встановлювати терміни виконання.
- *Kanban-дошки*: дозволяють візуально відображати завдання на різних етапах виконання. Це спрощує управління робочими процесами та допомагає командам слідкувати за прогресом;
- *Milestones*: інструмент для визначення ключових етапів проєкту. Команди можуть використовувати його для фіксації дедлайнів та пріоритизації завдань;
- *інструменти звітності*: сервіси надають аналітичні дані про виконання завдань, трекінг часу та ефективність команди.

Використання даних інструментів дає змогу поєднувати розробку коду та управління проєктами в одній платформі, що спрощує процеси планування й виконання завдань, а також забезпечує прозорість для всіх учасників.

Вибір системи контролю версій залежить від потреб команди, її робочого процесу та технічних вимог. GitLab, GitHub та Bitbucket є найпопулярнішими платформами для управління репозиторіями, співпраці над кодом та інтеграції з

DevOps-практиками. Кожна з цих платформ має свої переваги та обмеження, які роблять їх більш або менш придатними для різних сценаріїв використання.

Відмінності між системами контролю версій

Критерій	GitLab	GitHub	Bitbucket
Модель використання	SaaS (хмара) і Self-Managed (локальний сервер)	Переважно SaaS, обмежені можливості локального розгортання (GitHub Enterprise Server)	SaaS і Self-Managed (Bitbucket Server)
CI/CD	Вбудований CI/CD без додаткових інструментів	GitHub Actions інтегрований, але вимагає окремого налаштування	Bitbucket Pipelines доступний, але обмежений для локальних серверів
DevSecOps	Вбудовані інструменти для безпеки: SAST, DAST, Dependency Scanning, Container Scanning	Обмежена підтримка, переважно через сторонні інтеграції	Обмежена підтримка, інтеграції через сторонні інструменти
Інструменти моніторингу	Вбудована інтеграція з Prometheus, Grafana для збору метрик і аналізу продуктивності	Немає власних інструментів моніторингу, доступ через сторонні рішення	Немає вбудованих інструментів моніторингу
Підтримка репозиторіїв	Необмежена кількість приватних і публічних репозиторіїв	Безкоштовні приватні репозиторії, з можливістю їх масштабування	Переважно підтримує приватні репозиторії (обмежені публічні)
Управління проєктами	Підтримка епік, дорожніх карт, Kanban-дошок, завдань і milestone	Основний фокус на Issues, Projects і Kanban-дошках (простий функціонал)	Інтеграція з Jira для розширених функцій управління проєктами
Вартість	Безкоштовний рівень із розширеними можливостями, платні тарифи з додатковими функціями	Безкоштовний рівень, але функції Enterprise доступні тільки у платних тарифах	Доступні гнучкі ціни, але платформи Atlassian (наприклад, Jira) часто вимагають додаткових витрат

GitLab більше підходить для комплексного управління DevOps-процесами завдяки вбудованому CI/CD, DevSecOps функціям і широкій інтеграції з DevOps інструментами.

GitHub ідеально підходить для відкритих проєктів і команд, що шукають потужний інструмент із широким вибором інтеграцій і GitHub Actions для CI/CD.

Bitbucket найкраще інтегрується з продуктами Atlassian, такими як Jira, що робить його зручним вибором для команд, які використовують Atlassian Suite.